

- Users will never enter negative values.

When you say these statements out loud, they suddenly sound risky. Explanation makes hidden assumptions visible.

It improves problem

understanding: Sometimes the issue is not in the code — it is in misunderstanding the requirement.

By clearly explaining expected behaviour, you may discover:

- The requirement was incomplete.
- Edge cases were not considered.
- Failure scenarios were ignored.

Clarity in understanding significantly reduces debugging time.

From rubber duck to AI

A physical rubber duck listens silently. AI, however, can:

- Ask clarifying questions.
- Reflect your logic back to you.
- Point out missing scenarios.
- Simulate edge cases.

But there is an important difference. If you use AI only to generate solutions, you reduce your own growth. If you use AI to test and refine your thinking, you become stronger. The goal is not faster answers. The goal is clearer reasoning.

How to use AI as a rubber duck

Here is a structured approach suitable for students, working engineers, and system architects.

Step 1: Define the role clearly:

Before sharing code, guide the AI.

Prompt example: Act as a rubber duck debugger. Do not immediately give me a solution. Listen to my

explanation and point out unclear logic, missing steps, or wrong assumptions.

This keeps the focus on reasoning rather than quick fixes.

Step 2: Explain the intended behaviour (not the code): Start with what the system should do.

Prompt example: I am writing a function to calculate discounts.

- > Premium users get 20%.
- > Regular users get 10% if purchase is above ₹5000.
- > Others get no discount.

Immediately, important

questions arise:

- What if the purchase is exactly ₹5000?
- What if the amount is negative?
- Can a user be both premium and regular?
- What if the user type is missing?

Clarity begins before the first line of code is written.

Step 3: Walk through the logic sequentially: Now explain your implementation step by step.

Prompt example: Here is my step-by-step logic:

1. Check if the user is premium.
2. If yes, apply 20%.
3. Else check if purchase > 5000.
4. Apply 10%.
5. Return final amount.

Please restate this and tell me if any case is not handled.

AI may highlight:

- Missing boundary condition

(exactly 5000).

- Lack of validation for negative input.
 - Overlapping conditions.
- This improves completeness and robustness.

Step 4: Identify assumptions: This is the most critical stage.

Prompt example: What assumptions am I making here? Under what real-world situations can this logic fail?

Step 5: Explore edge cases: Most production failures occur in rare situations.

Prompt example: List possible edge cases for this logic.

This step prepares your system for real-world unpredictability.

Step 6: Request improvement: This is the final step once there is clarity and reasoning is clear.

Prompt example: Based on our discussion, suggest a clean and safe implementation.

In this discussion AI supports your thinking instead of just replacing it.

Technology keeps evolving. AI tools are becoming more powerful every year. But one thing remains constant — software quality depends on clarity of thought.

Whether you use a physical duck, a colleague, a whiteboard, or an AI tool — the principle remains the same. Clear explanation leads to clear thinking. Clear thinking leads to better engineering. **END** 🐧

By: Anil Dhaneshwar

The author is a senior principal software engineer in InfoScale at Pune, Maharashtra. He has expertise in the Linux kernel, C, storage and system-level development.

THE COMPLETE MAGAZINE ON OPEN SOURCE

OpenSource
ForYou

The latest from the Open Source world is here.

OpenSourceForU.com

Join the community at facebook.com/opensourceforu

Follow us on Twitter @OpenSourceForU